Autonomous Systems - CSL7650

Assignment - 2

IIT Jodhpur Academic Helpdesk: A Multi-Agent Retrieval-Augmented Assistant

Source Code: https://github.com/sarahfatima1205/Institute-RAG-system

Sarah Fatima (B23EE1093) Sahil Narkhede (B23CS1060)

Anshit Agarwal (B23CS1087) Namya Dhingra (B23CS1040)

October 29, 2026

Abstract

Students frequently need answers to questions about academic regulations, course timetables and administrative procedures, but the relevant rules are spread across regulation documents, amendment circulars and course lists, and are revised over time. This project builds a local academic helpdesk for IIT Jodhpur that answers such questions only from official documents and course data, cites its sources, and drafts (but never sends) leave and make-up examination emails to instructors. The system uses Google ADK agents served by local Ollama models: an orchestrator routes each question to a policy, schedule or email specialist, which retrieve evidence from a ChromaDB index of the regulation documents or from the course CSV. Newer amendments automatically supersede the older rules they replace, and questions that cannot be answered from the sources are refused and logged for human follow-up. The multi-agent system with Qwen3 8B is compared against a single-agent baseline with Llama 3.1 8B on core prompts covering policy, timetable, email and out-of-scope questions. The multi-agent configuration answered all 15 evaluated prompts correctly with 91% citation groundedness, against 87% accuracy and 36% groundedness for the baseline, and both configurations correctly refused every out-of-scope question.

Contents

1 Project overview **3**

2 Assignment scope and coverage **3**

3 System design **4**

3.1 Agent architecture 4

3.2 Request and response flow 4

3.3 Retrieval-augmented generation 4

3.4 Tools 5

4 Data and document corpus **5**

4.1 Corpus gaps and data work needed 5

5 User interfaces and execution **6**

6 Evaluation design **6**

7 Results **7**

7.1 Setup 7

7.2 Overall performance 7

7.3 Performance by question category 8

7.4 Response time 10

8 Current limitations and recommendations **10**

9 Conclusion **11**

10 References **11**

1 Project overview

This project implements an academic helpdesk that answers questions about IIT Jodhpur academic rules, programmes, curriculum, courses and timetable data. It can also prepare (but does not send) email drafts to course instructors for leave, sick leave and make-up examination requests.

The helpdesk is intended to answer only from the project's documents and course data. For out-of-scope questions or questions it cannot support with available sources, the agents are instructed to say "I don't know" and log the question for human follow-up. Factual answers are expected to cite the source document or CSV.

The project is implemented in Python with Google ADK agents, LiteLLM's Ollama connector, Ollama-hosted chat and embedding models, and ChromaDB for local vector retrieval. The default chat model is `qwen3:8b`; `llama3.1:8b` can also be selected. A single-agent baseline is provided alongside the multi-agent system.

2 Assignment scope and coverage

Assignment requirement	**Project implementation**
Answer academic policy and curriculum questions	Policy specialist searches the indexed academic documents.
Answer course, slot and timetable questions	Schedule specialist searches `courses.csv` by code/name and filters or counts rows.
Draft leave and make-up examination emails	Email specialist resolves course/instructor information and creates a reviewable draft. No email is sent.
Refuse unrelated or unsupported questions	Prompt rules and retrieval thresholds direct the system to refuse; the refusal workflow logs unanswered questions.
Cite source material	Agent instructions require citations; retrieval results provide document/section citations and course tools identify `courses.csv` as their source.
Use a local setup suitable for a 16 GB computer	Local Ollama models and a persistent local ChromaDB are configured. The evaluation instructions specify 16 GB RAM as the minimum for the 8B models.
Compare single/multi-agent and model performance	The evaluation harness supports single and multi-agent runs, multiple models, repeated runs, and comparisons.
Collect 20–30 real source documents	**Not yet met:** the current document corpus has 14 files. The timetable is also sample data, not a real semester timetable.

3 System design

3.1 Agent architecture

The multi-agent system is organized around a helpdesk orchestrator and three specialist agents:

- **Policy agent:** handles regulations, attendance, examinations, grading, credits, degree requirements, programmes, curriculum and the academic calendar. It uses document search and unanswered-question logging.
- **Schedule agent:** handles course codes, names, slots, venues, credits, course levels, types and filtered course counts using the course CSV.
- **Email agent:** resolves a course and its instructor, then drafts the requested email and attaches a relevant policy note. Unknown course codes are not guessed.
- **Helpdesk orchestrator:** routes academic questions to a specialist and refuses clearly unrelated questions.

The single-agent baseline has access to all tools directly. It supports comparisons with the orchestrated system.

3.2 Request and response flow

1. A student submits a question through the terminal or web interface.
2. The helpdesk sends the question to the selected ADK agent configuration.
3. The agent calls the appropriate RAG or CSV/email tool.
4. The response is composed from returned tool results, with source citations required by the instructions.
5. If the question is unsupported or out of scope, the agent should respond with "I don't know" and record the question in a CSV log.

3.3 Retrieval-augmented generation

The RAG pipeline loads PDF, HTML, Markdown and text documents, splits content into sections and overlapping chunks, embeds the chunks, and persists them in ChromaDB. Ollama's `nomic-embed-text` is the configured local embedding model. The pipeline retrieves a candidate set using cosine similarity, then re-ranks candidates using keyword overlap.

The configuration includes a minimum similarity threshold, a candidate count, a top-k result count, chunk size and overlap. Documents carry source and effective-date metadata. Markdown amendments can declare which older rules they supersede; matching older passages are withheld and the newer amendment is returned with a current-rule note.

The `hash` embedding backend is reserved for offline unit tests and is not the production semantic embedding configuration.

3.4 Tools

- `search_policy_documents`: retrieves relevant policy passages with citations and effective-date information.
- `find_course`: searches course codes and names, including a limited fuzzy name match.
- `list_courses`: filters and counts course rows by department, level, course type and slot.
- `get_instructor_email`: returns the instructor contact listed for a course.
- `draft_email`: creates a leave, sick-leave or make-up examination email, leaves placeholders for unknown student details, includes a policy note, and marks the result as an unsent draft.
- `log_unanswered_question`: appends the timestamp, question and reason to the unanswered-question CSV.

4 Data and document corpus

The manifest currently lists 14 document files covering:

- Undergraduate and postgraduate regulations, including earlier regulation editions.
- The official academic regulations page.
- Attendance and examination amendments from 2024–25. The PDF circulars are scans; their Markdown transcriptions are used for indexing.
- An academic calendar for 2026–27.
- B.Tech. CSE curriculum and CSE course details.
- M.S. by Research regulations.

The document manifest records categories, source URLs, effective dates and status notes. The RAG loader uses this metadata to provide source details and to support newer-rule precedence.

The timetable data is in `data/csv/courses.csv`. Its accompanying README identifies it as **sample data** with placeholder course rows and dummy instructor names and email addresses. It is suitable for exercising the tools, but should not be treated as an authoritative live timetable or real faculty directory.

4.1 Corpus gaps and data work needed

The assignment requests 20–30 real documents. The current manifest has 14 document files, leaving the corpus below that target. The project README identifies potential additions such as current timetable and slotting information, add/drop and withdrawal circulars, make-up or supplementary exam notices, grading notices, minor/specialisation rules, additional departmental curricula and an anonymised faculty email list.

Before using the system as an authoritative helpdesk, replace the sample timetable with a verified current source, retain dummy or anonymised contacts where appropriate, verify document dates and applicability, and add enough relevant source documents to meet the assignment target.

5 User interfaces and execution

The project provides:

- **Command-line chat:** `main.py`, with `-mode single|multi`, optional `-model`, and `-ingest` to build the vector index before starting.
- **Web chat:** `web.py`, serving a local chat page and maintaining chat history. It uses the multi-agent configuration and does not ingest documents automatically at startup.
- **Indexing:** `rag.py`, which builds or rebuilds the persistent ChromaDB collection from the configured document directory.

The README documents setup using a Python virtual environment, installing `requirements.txt`, obtaining the Ollama models, and building the vector index. The default model endpoint is local Ollama; `CHAT_MODEL`, `OLLAMA_API_BASE`, and other retrieval settings can be configured through environment variables.

The project is designed to run locally without hosted model API keys. The evaluation instructions specify Python 3.11 or later and 16 GB RAM as the minimum for running an 8B model; actual response times depend on the machine and model.

6 Evaluation design

The evaluation set in `eval/prompts.json` contains 73 prompts:

- **30 core prompts:** 13 policy, 6 schedule, 4 email and 7 out-of-scope questions.
- **43 extended prompts:** paraphrases, near-domain traps, typos, mixed-language input, adversarial instructions and multi-turn follow-ups.

Prompts include expected-answer patterns, rejected patterns where needed, permissible source names, refusal expectations and, for multi-agent runs, expected specialist routes. The evaluation script computes:

- **Accuracy:** expected patterns match, rejected patterns do not, and the system has not refused an in-scope question.
- **Groundedness:** a cited source is among the expected sources and was returned by a tool during that run.
- **Correct refusal:** the answer indicates that it does not know and the unanswered-question logging tool was called.
- **Routing:** the expected specialist participated in a multi-agent run.
- **Stability:** answer text and pass/fail behavior across repeated runs.
- **Response time:** per-answer duration and category averages.

The harness supports repeated runs, resuming interrupted evaluations, filtering by prompt set or ID, and comparing two result files. `EVAL_STEPS.md` describes a four-configuration comparison: multi-agent Qwen3 8B, single-agent Qwen3 8B, multi-agent Llama 3.1 8B and single-agent Llama 3.1 8B. It recommends three runs for the 30 core prompts and one run for the extended prompts.

The unit tests focus on offline retrieval and amendment handling, course lookup/count behavior, email drafting, citations/source behavior and unanswered-question logging. They use the hash embedder and do not require a live Ollama model.

7 Results

7.1 Setup

Both configurations were evaluated on the same machine and the same prompts, one configuration computing at a time so that response times are comparable.

Hardware	Intel Core i7-1260P (12 cores, 16 threads), 31 GB RAM, CPU only
Model server	Ollama 0.40.2, 8192-token context, `nomic-embed-text` embeddings
Configurations	Multi-agent with `qwen3:8b`; single agent with `llama3.1:8b`
Prompts	15 core prompts (4 policy, 4 timetable, 4 email, 3 out of scope), one run per configuration
Index	1090 chunks (9 superseded by the 2024 amendments)

Table 2: Evaluation setup.

7.2 Overall performance

Table 3 and Figure 1 summarise the results. The multi-agent Qwen3 8B configuration answered all 15 prompts correctly, routed every in-scope question to the expected specialist, and cited a source that its tools had actually returned in 10 of 11 in-scope answers. The single-agent Llama 3.1 8B baseline reached 87% accuracy, but cited a valid source in only 4 of 11 answers: it frequently gave a correct answer without any citation. Both configurations refused and logged all four questions that could not be answered from the sources.

Configuration	Accuracy	Groundedness	Correct refusal	Routing	Mean s/answer
Multi-agent, Qwen3-8B	15/15 (100%)	10/11 (91%)	4/4 (100%)	11/11 (100%)	70
Single agent, Llama-3.1-8B	13/15 (87%)	4/11 (36%)	4/4 (100%)	–	64

Table 3: Overall results. Routing applies to the multi-agent configuration only.

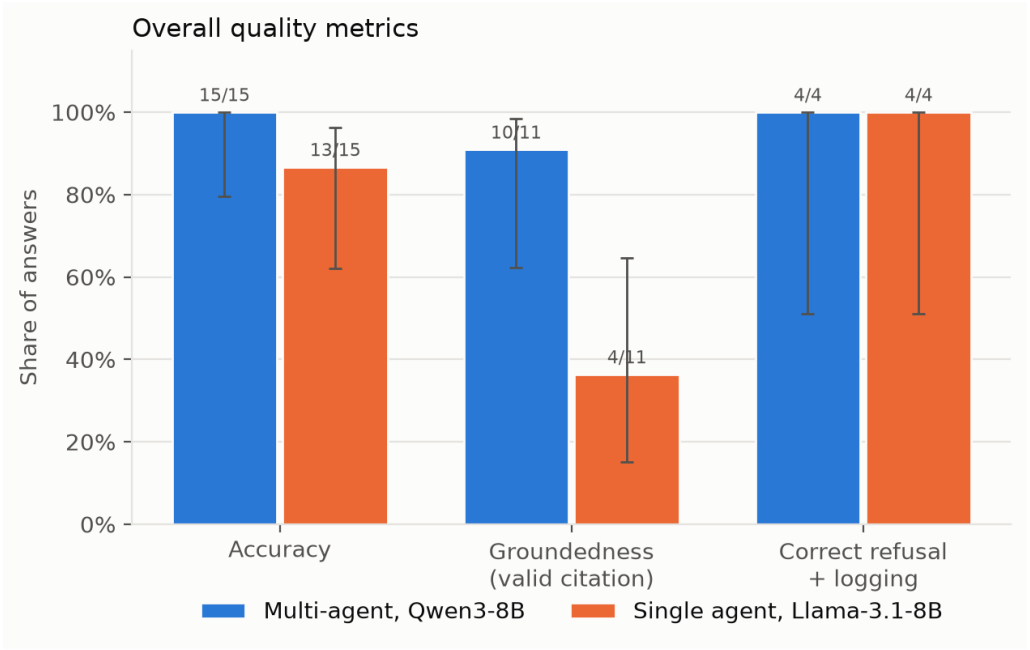


Figure 1: Accuracy, groundedness and correct refusal by configuration (error bars: 95% Wilson score interval).

7.3 Performance by question category

Table 4 and Figure 2 break accuracy down by category. Both configurations answered every timetable and out-of-scope prompt correctly; the differences lie in the policy and email questions, which require document retrieval or a correct course lookup before drafting.

Category	Prompts	Multi-agent, Qwen3-8B	Single agent, Llama-3.1-8B
Policy	4	4/4 (100%)	3/4 (75%)
Timetable	4	4/4 (100%)	4/4 (100%)
Email	4	4/4 (100%)	3/4 (75%)
Out of scope	3	3/3 (100%)	3/3 (100%)

Table 4: Accuracy by question category.

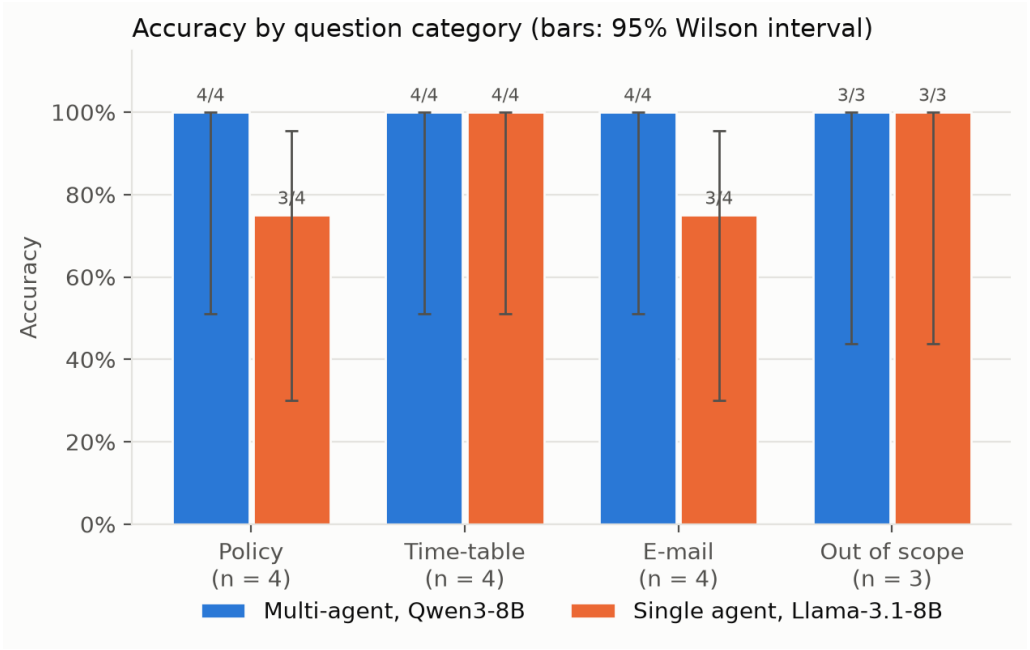


Figure 2: Accuracy by question category (error bars: 95% Wilson score interval).

Figure 3 shows the result of every prompt. The baseline failed two prompts. For P02 (Open Elective credits) it retrieved a passage from the 2018 B.Tech. regulations and reported a 180-credit total with a derived breakdown that does not appear in the documents, instead of the 10 Open Elective credits stated in the current regulations. For E02 it drafted the make-up request without first looking up "Autonomous Systems", addressing the e-mail to the instructor of a different course. The multi-agent system avoids the second error because its email specialist is instructed to resolve the course before drafting.

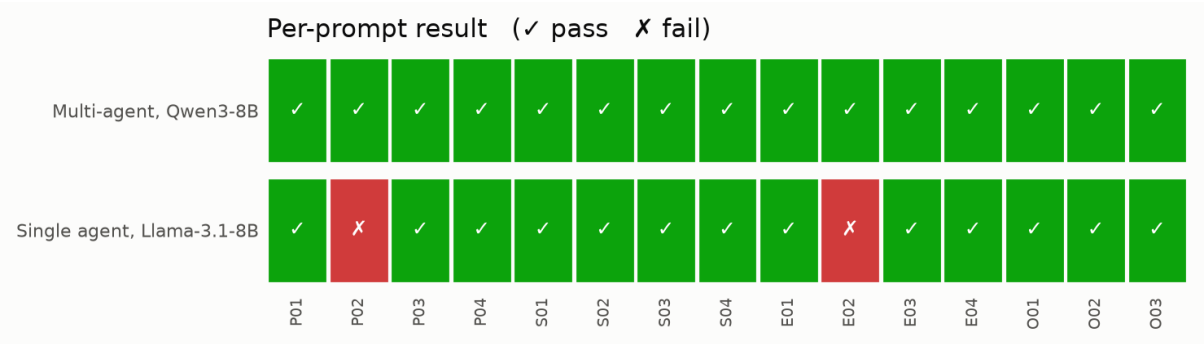


Figure 3: Pass/fail result per prompt for both configurations.

The single ungrounded answer of the multi-agent system (P04) gave the correct ten-day make-up rule but cited the PG regulations for an undergraduate question, so it does not count as grounded. The baseline's ungrounded answers were mostly correct timetable and email answers that omitted the `(Source: ...)` citation required by the instructions.

7.4 Response time

Figure 4 compares response times. The multi-agent system needs an extra model call for routing, which makes it slower on timetable and email questions (38 s vs. 18 s and 99 s vs. 77 s), but it is much faster on out-of-scope questions (16 s vs. 55 s) because the orchestrator refuses them directly without calling a specialist. Policy questions are the slowest for both configurations (113 s and 104 s), as they require retrieval and longer answers.

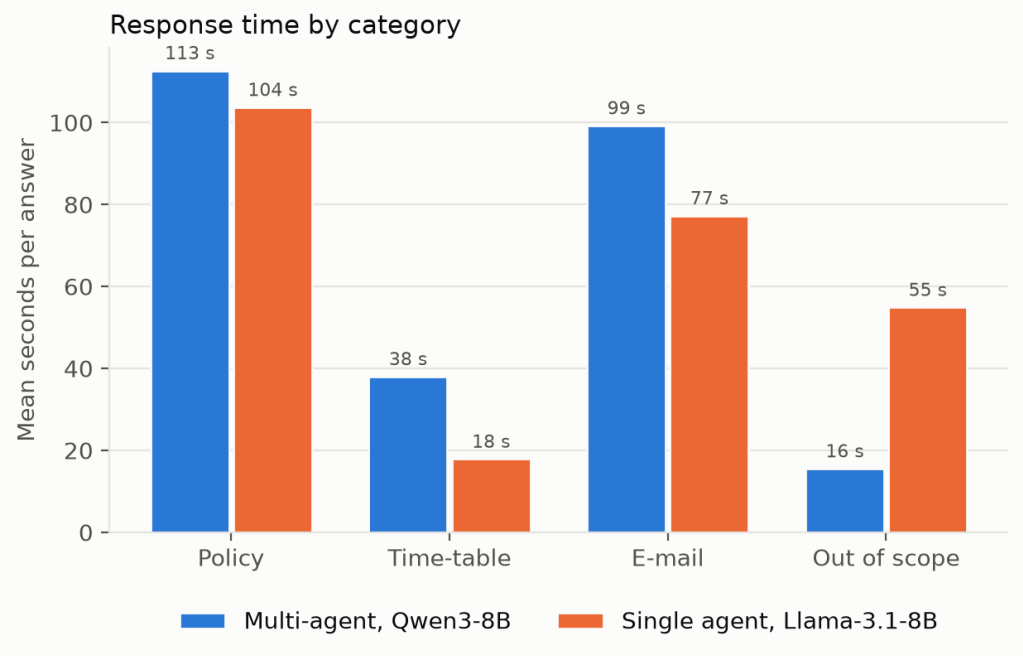


Figure 4: Mean response time per answer by category (CPU).

8 Current limitations and recommendations

1. **Complete the source collection.** Add at least six further real, relevant documents to reach the assignment's minimum of 20, and preferably build toward 30. Track provenance and effective dates in the manifest.
2. **Replace sample timetable data.** Use a current, verified timetable. Update prompt gold answers to match the real data and retain anonymised contact details if needed.
3. **Extend the evaluation.** Run all 30 core prompts with three repeated runs per configuration, the 43 extended prompts, and the two remaining configurations (single-agent Qwen3 8B and multi-agent Llama 3.1 8B) to measure run-to-run stability.
4. **Review citation and routing reliability.** The README notes that 8B models may occasionally paraphrase citations or route borderline questions incorrectly. Validate these behaviors using the groundedness and route metrics.
5. **Validate policy applicability.** The corpus contains documents for different admission cohorts and programmes. The system should be evaluated to ensure answers distinguish the applicable cohort and do not blend conflicting rules.
6. **Keep the index current.** Rebuild the vector store when indexed documents change, and

periodically verify that downloaded or transcribed documents still match their official sources.

9 Conclusion

The project provides a working architecture for a local, multi-agent academic helpdesk, including policy RAG, timetable tools, draft email generation, out-of-scope logging, user interfaces and an evaluation harness. The code covers the core functional shape of the assignment. In the evaluation, the multi-agent Qwen3 8B configuration answered all 15 evaluated core prompts correctly with 91% groundedness, compared with 87% accuracy and 36% groundedness for the single-agent Llama 3.1 8B baseline; both configurations correctly refused and logged all out-of-scope prompts. The main remaining assignment gaps are the undersized document corpus and placeholder timetable/contact data.

10 References

[1] Google, "Agent Development Kit (ADK) Documentation," Google Cloud. Available: `https://google.github.io/adk-docs/`

[2] Qwen Team, "Qwen3 Technical Report," 2025. Available: `https://arxiv.org/abs/2505.09388`

[3] The Llama Team, "The Llama 3 Herd of Models," 2024. Available: `https://arxiv.org/abs/2407.21783`

[4] Chroma, "Chroma Documentation: Open-Source Vector Database for AI Applications." Available: `https://docs.trychroma.com/`

[5] Ollama, "Ollama Documentation: Get Up and Running with Large Language Models." Available: `https://docs.ollama.com/`

[6] Hugging Face, "Sentence Transformers: Text Embeddings and Semantic Search." Available: `https://www.sbert.net/`

[7] LiteLLM, "LiteLLM Documentation: Unified Interface for Large Language Models." Available: `https://docs.litellm.ai/`